# Automating Financial Account Creation

### An R Pipeline for SOP-to-Database Reporting at JBS Australia

Andy Tran

2026-06-01

**Git Repository:** https://github.com/Jakey-vn/ETC5543_Project

**Generative AI Acknowledgement:** Claude (Anthropic) was used to assist with structuring and drafting sections of this report, as well as debugging R code during development. All analysis, design decisions, and conclusions are made without anyone nor AI's assistance.

---

## Introduction

## About JBS

JBS N.V. is a Brazilian multinational founded in 1953 in Anápolis, Goiás, and is the world's largest meat processing enterprise by revenue. Headquartered in São Paulo and Amsterdam, the company processes beef, pork, chicken, lamb, and salmon. As of 2025, JBS operates more than 250 production facilities worldwide, with its products reaching consumers in 180 countries. In Australia, the company operates multiple beef, pork, and lamb processing facilities under several distinct business divisions.

## JBS Australia Pork (Rivalea)

JBS Pork Australia, formerly known as Rivalea is the pork division of JBS in Australia, and the host organisation for this internship. Operating across Victoria and New South Wales, it is the country's leading integrated pork producer, accounting for approximately 15% of Australia's total domestic pork production and around 26% of the local fresh pork market share. The division supports high-quality pork production for both domestic and export markets through extensive agricultural, processing, and marketing operations.

## The Finance Team

The Finance Team, where the internship take place, is responsible for generating financial reports on a weekly and monthly basis, covering two processing sites operated by JBS Pork Australia:

- **Corowa** (New South Wales): the primary site, covering Farming Operations, Kill Floor, Boning Room, Boxed Meat, Offal, Commodity Bone, and Meat Trade
- **Laverton** (Victoria): a secondary site handling Kill Floor, Boning Room, Boxed Meat, Offal, Commodity (chilled and frozen), and Sow Bone

Across these two sites, financial account entries must be generated for each reporting period. These entries capture revenue (sales of various product categories) and expenses (packaging, trade allowances, storage, and freight), broken down by Business Unit (BU), GL account, Cost Center, and month.

## Key Terms

The following terms appear throughout this report. Readers from outside financial accounting or meat processing may find these definitions useful.

### Financial Terminology

**General Ledger (GL) Account** - A numbered account in the company's accounting system that categorises every financial transaction. Revenue from selling Boxed Meat, for example, posts to a different GL account than the expense for packaging materials. The GL account determines *what* a transaction represents in the accounts.

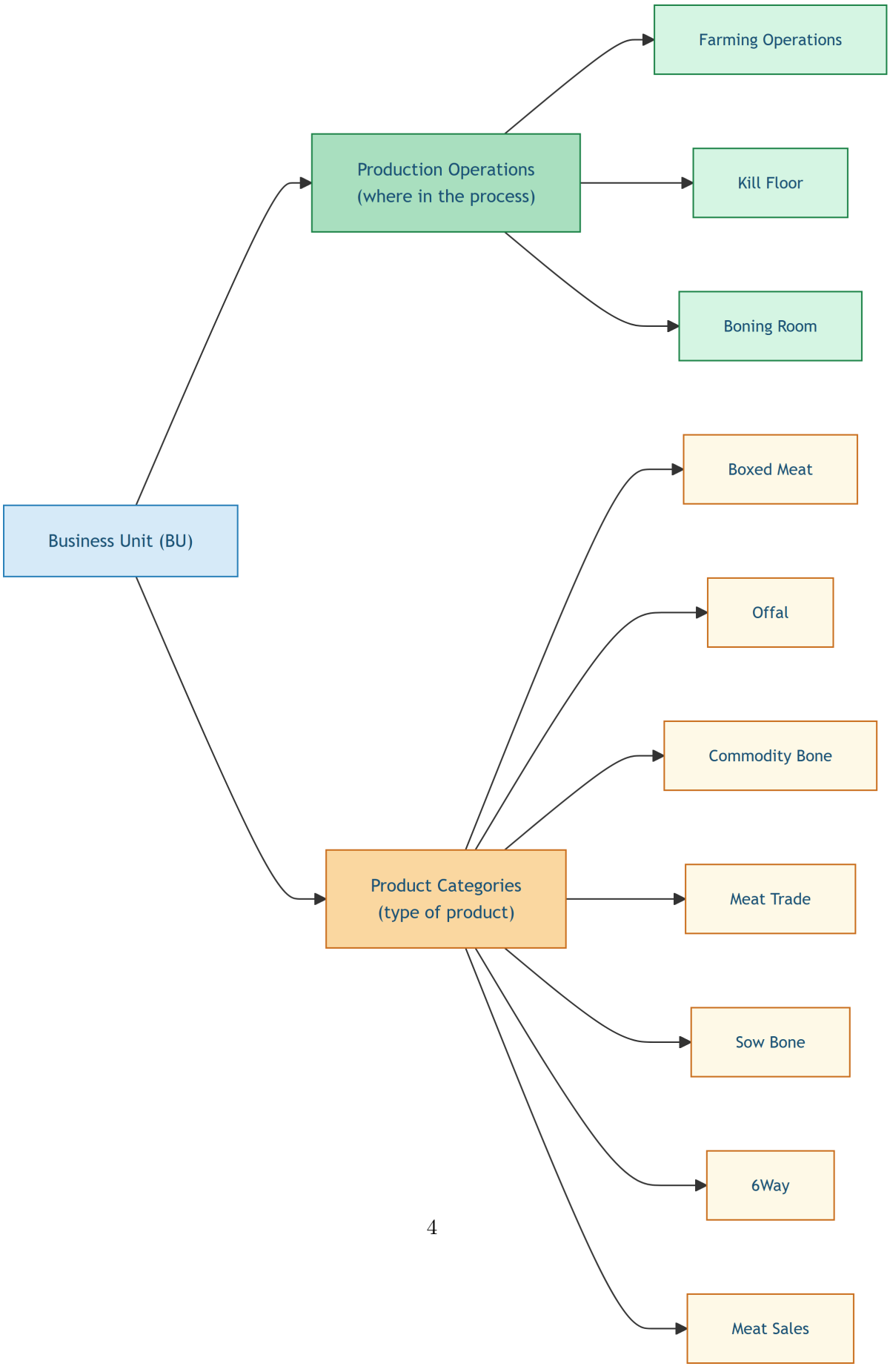
**Cost Center (CC)** - A code that identifies *where* a cost or revenue belongs within the organisation - for example, the Boning Room at Corowa versus the Kill Floor at Laverton. GL account and Cost Center together form the two-part key used to route every database row to the correct place in the financial system.

**Business Unit (BU)** - A distinct operational segment of the business. Each BU has its own set of GL accounts and Cost Centers, and the pipeline generates a separate block of database rows for each one.

**Value-Add (VA)** - Products that undergo an additional processing step to increase their market value beyond the base cut. A marinated pork loin is a typical example: the raw loin is transformed into a higher-value retail product through seasoning and packaging. VA products are reported under the Meat Sales Business Unit.

### Business Units

Each processing site is divided into Business Units that fall into two categories: **production operations** (stages in the physical processing chain) and **product categories** (the type of product being reported on).


Business Unit (BU)
Production Operations
(where in the process)
Farming Operations
Kill Floor
Boning Room
Product Categories
(type of product)
Boxed Meat
Offal
Commodity Bone
Meat Trade
Sow Bone
6Way
Meat Sales

| Type | Business Unit | Description |
|---|---|---|
| Production Operation | Farming Operations | Covers the farm stage of the supply chain - livestock rearing, feed costs, and farm-level expenses before pigs reach the processing facility |
| Production Operation | Kill Floor | The slaughter and primary carcass breakdown stage |
| Production Operation | Boning Room | Separation of meat from bone after slaughter |
| Product Category | Boxed Meat | Portioned and vacuum-packed cuts prepared for sale |
| Product Category | Offal | Co-products such as organs and secondary cuts |
| Product Category | Commodity Bone | Bone material sold as a commodity, either chilled or frozen |
| Product Category | Meat Sales | Value-Add (VA) products sold with revenue; see *VA* in Financial Terminology above |
| Product Category | Meat Trade | Represents meat volume transferred from Laverton to Corowa for further processing or sale |
| Product Category | Sow Bone | Bone from sow (female pig) carcasses; Laverton site only |
| Product Category | 6Way | A pork carcass cut into six standard portions; revenue is tracked as a distinct product line |

### Supply Chain and Cost Terms

**SOP Team (Sales & Operations Planning)** - The team responsible for forecasting production volumes and coordinating supply with demand. Their weekly SOP Volumes spreadsheet is the primary source of volume data for this pipeline.

**Trade Discounts** - Negotiated price reductions applied to specific customers or product lines. Rather than reducing the sale price directly, these are recorded as a separate expense line in

the database.

**Packaging Costs** - The cost of materials used to pack finished product (vacuum bags, cartons, labels). These are calculated per kilogram of product and assigned as an expense account entry.

**Storage Rates** - Charges for holding product in chilled or frozen storage before dispatch. Applied to Commodity Bone and certain other product lines based on volume.

**Freight** - The cost of transporting product from the processing site to the customer. Rates vary by product line and are sourced from the Rates & Allocation file.

### Report Structure

Section 2 provides background on the problem, the previous Excel-based solution, the rationale for choosing R, and the five input data sources. Section 3 describes the pipeline architecture and calculation methodology for both revenue and expense accounts. Section 4 documents the Gap Check quality assurance tool. Section 5 evaluates the pipeline against the manual process. Section 6 discusses current limitations and future improvements. Section 7 concludes.

---

## Background

### The Problem

The Finance Team's goal was to build a monthly financial report that could be refreshed each period with updated values, giving managers a clear picture of all products, categories, and revenue across both sites. The intended output was a structured database that anyone could query to retrieve information such as which products were produced at Corowa, what revenue they generated, and which General Ledger (GL) account and Cost Center they belonged to.

Building this report required bringing together data from multiple sources:

1. Volume data from the SOP Volumes spreadsheet, sourced from the S&OP team
2. Prices maintained by the Sales team and stored in a separate binary file (`Prices.xlsb`)
3. GL and Cost Center codes assigned from a Master Data file maintained by NAM and Commercial Finance
4. Trade discounts, packaging costs, storage rates, and freight charges from two additional files provided by the Sales team and Business Planning Manager

## Previous Solution

Prior to this project, the Commercial Finance Manager had implemented the report in Excel. While functional, the workbook grew to more than 20 sheets to accommodate the full scope of accounts across two sites and multiple Business Units. This scale made the file difficult to navigate, verify, and maintain. The process was not reproducible: each new reporting period required significant manual re-work, and the transformation logic was spread implicitly across sheet formulas rather than documented in any single place.

The number of accounts involved compounded the risk. Two sites, multiple Business Units, twelve months, and numerous product categories each generate separate database rows. Any error like a missed row, an incorrect price lookup, or a copy-paste mistake would propagate into the financial database without automatic detection. Because the process had to be repeated every reporting period, this risk recurred each time.

## The R Pipeline Solution

To address these limitations, this project re-implemented the pipeline in R. Unlike the Excel workbook, an R script is plain text that can be version-controlled in Git: every change to the logic is recorded with a timestamp and author, creating a full audit trail. The pipeline is fully reproducible: running the script on any machine with access to the shared OneDrive folder will always produce the same output from the same source files.

The modular structure of the R code means adding a new Business Unit or adjusting a calculation requires editing one clearly-defined function rather than restructuring an entire workbook. A built-in quality assurance tool called "Gap Check" was developed to surface data integrity issues automatically before any output is submitted, replacing the manual spot-checks that the Excel approach required.

## Choice of Tools

**R** was selected as the primary language for this project for several practical reasons:

- JBS source files are Excel-based (`.xlsx` and `.xlsb`). The `readxl` and `openxlsx2` packages in R provide reliable, well-tested support for both formats, including multi-sheet workbooks and binary `.xlsb` files that other tools handle poorly.
- The `tidyverse` ecosystem, particularly `dplyr` and `tidyr` allows complex multi-file joins, filters, and transformations to be expressed in a clear, readable style that can be reviewed by colleagues without deep programming knowledge.
- `openxlsx` enables the pipeline to write formatted Excel output (the Gap Check report), which is the format most accessible to the finance team.

**Quarto** (`.qmd`) was used as the document format. Quarto allows R code and written documentation to coexist in a single file, so the pipeline logic and its explanation are maintained together. It renders to HTML for interactive review and integrates naturally with Git version control.

## Data Sources

### Overview of Input Files

The pipeline reads five files at runtime. All files are stored in a shared OneDrive folder, so the pipeline always uses the most current version without a manual download step.

Table 2: Pipeline input files and their roles

| File | Contents | Maintained By |
|---|---|---|
| SOP Volumes.xlsx | Weekly volumes by Farm / Bone / VA product lines | S&OP Team |
| Prices.xlsb | Carcass price, Kill fee, Bone fee, Item prices, Packaging rates | Sales Team |
| Master Data.xlsx | GL accounts, Cost Center keys, Customer list, VA brand mapping | NAM & Comm |
| Trade Allowance.xlsx | Trade discount rates per customer | Business Partn |
| Rates & Allocation.xlsx | VA brand allocations, freight rates, storage rates | Business Partn |

### SOP Volumes Structure

The SOP Volumes file contains a single sheet (`Sheet1`) with all product-customer-week-month combinations across both sites. A key column (column 2) identifies the product category type for each row:

- `Farm`, `Kill`, `Bone` - livestock and primary processing operations (Farm tab)
- `2.Meat Processing`, `6.Laverton Processing` - bone room and boxed meat (Bone tab)
- `3.Meat Sales` - value-added products (VA tab)

The file is loaded entirely as character text with no headers at import time, then split into three separate data frames based on this identifier column. This approach avoids type-parsing failures that arise from mixed numeric and text columns, which are common when R attempts to auto-detect column types in complex spreadsheets.

```
# Load raw volume as text - avoids type-parse errors on mixed columns
volume_raw <- read_excel(
  "C:/Users/ATran2/OneDrive - JBS/Forecast Project - Documents/SOP Volumes.xlsx",
  sheet    = "Sheet1",
```

```
  col_names = FALSE,
  col_types = "text"
)

# Split into three data frames by product category identifier
volume_farm_raw <- volume_raw %>% filter(`...2` %in% c("Farm", "Kill", "Bone"))
volume_bone_raw <- volume_raw %>% filter(`...2` %in% c("2.Meat Processing", "6.Laverton Proce
volume_va_raw   <- volume_raw %>% filter(`...2` == "3.Meat Sales")
```

### Data Quality Challenges

Source data quality was one of the most significant recurring challenges during development. Several categories of issues were encountered:

**Inconsistent naming.** Customer and product names in SOP Volumes contained trailing whitespace, inconsistent capitalisation, and occasional typos. These caused silent join failures - rows that should have matched a Master Data entry produced `NA` instead and were lost from the output. The solution was to apply `str_trim()` and `toupper()` to all key join columns before performing any merge.

**Missing price entries.** `Prices.xlsb` frequently lacked entries for newer item codes not yet added to the price reference sheets. Revenue rows for those items would produce `NA` values without any visible warning. This issue directly motivated the design of the Gap Check (Section 5).

**Missing classification fields.** Commodity Bone rows require a `Storage` field (`CHILLED` or `FROZEN`) to route to the correct account. When this field was blank or contained an unrecognised value, those rows could not be correctly categorised and were excluded from the output.

**Incomplete Master Data.** If a customer or GL account entry was absent from Master Data, the corresponding volume rows had no account to join to and were silently dropped. The Gap Check reconciliation table was designed to make this visible.

---

## Pipeline Design and Methodology

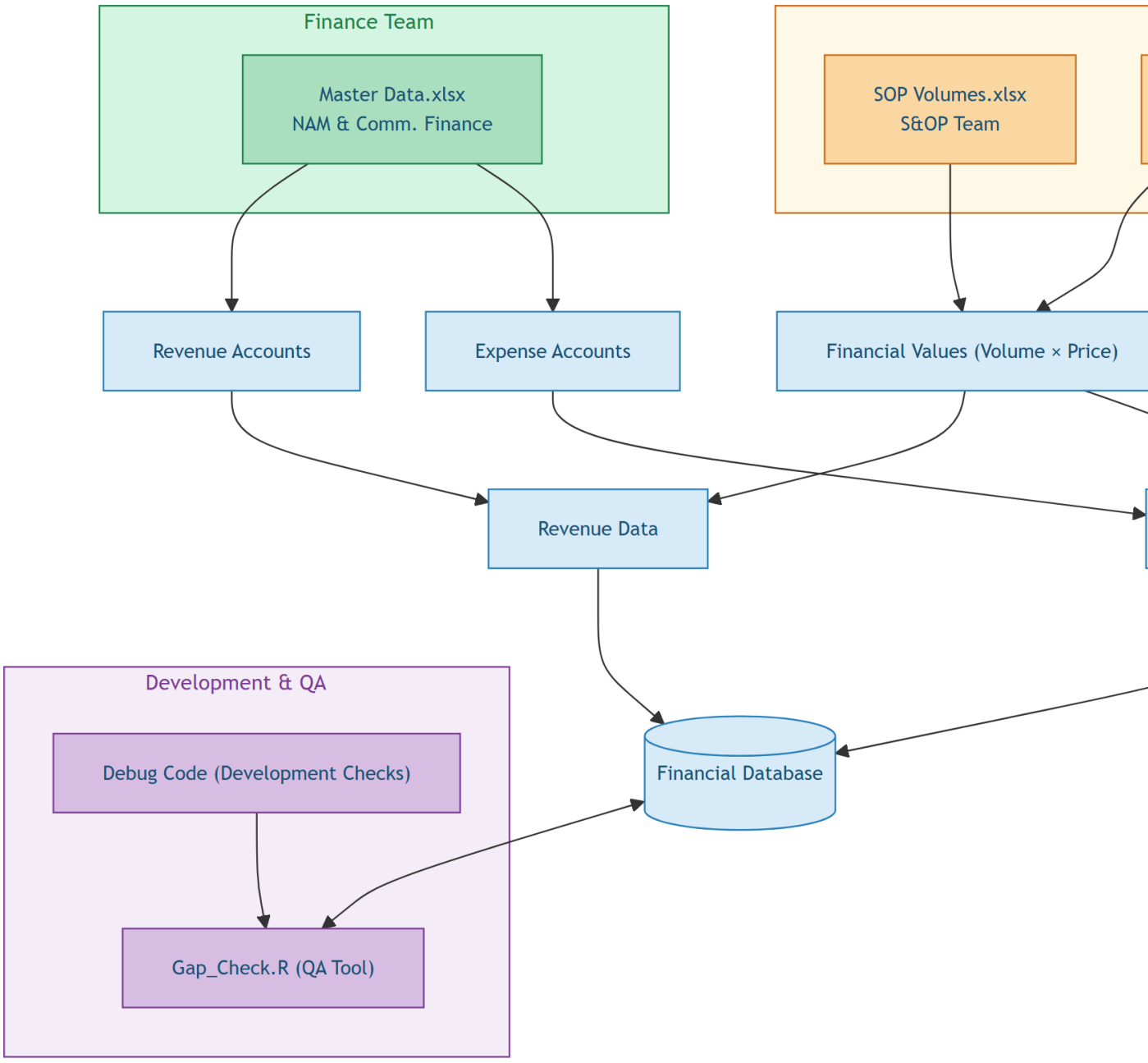
### Architecture Overview

The pipeline follows a three-stage pattern that is applied consistently to every Business Unit:

1. **Build account skeleton** - Pull all relevant customers from Master Data and cross them with the correct GL account and Cost Center key, then expand across all 12 months using `crossing(Period = 1:12)`. Building the skeleton first ensures that months with zero volume still produce a row in the database, rather than being silently absent from the output.
2. **Join SOP volume** - Match source volume rows from SOP Volumes onto the skeleton using a composite key (SOP Unicode or Item Code + Abattoir + Month + Tab). A left join is used so that skeleton rows with no matching volume retain a zero rather than being dropped - this is important for completeness in the final database.
3. **Calculate financial value** - Multiply the joined volume by the appropriate rate to produce a dollar value for each row. The rate source differs by tab type: Carcass Price per kg for Farm rows, Kill Fee per head for Kill rows, and Bone Fee per head for Bone rows. For expense accounts, rates are joined from Prices.xlsb, Trade Allowance, or Rates & Allocation depending on the expense category.

As a concrete example: for Farming Operations at Corowa, the skeleton contains all Corowa farm customers with GL 400000, replicated across 12 months. SOP volume rows for January are matched to their skeleton counterparts, and the resulting weight is multiplied by the carcass price to produce the January revenue value for each customer.

The diagram below shows how the five source files interconnect at the system level, and how data flows from raw inputs through revenue and expense calculations into the final financial database.




## Data Ingestion and Normalisation

All five files are read at the start of the pipeline. Normalisation is applied immediately after reading, before any joins are performed. The key normalisation steps are:

```r
# Normalise key columns before any joins are attempted
bone_base <- volume_bone_raw %>%
  mutate(
    `Item Code`    = str_trim(`Item Code`),
    Run            = toupper(Run),
    Type           = toupper(Type),
    Storage        = toupper(Storage),
    Abattoir       = toupper(Abattoir),
    Month          = toupper(Month),
    Tab            = toupper(Tab),
    `Total Weight` = as.numeric(`Total Weight`)
  )
```

The `BU` naming in `gl_cc_data` also required cleaning: the raw file contained inconsistent spacing after numeric prefixes (e.g., `"1. Farming"` vs `"1.Farming"`), which would silently break the skeleton join. A regex substitution resolved this:

```r
gl_cc_data <- gl_cc_data %>%
  mutate(BU = str_replace_all(BU, "(\\d+\\.)\\s+", "\\1"))
```

## GL Account Reference

The pipeline writes to thirteen distinct GL accounts across revenue and expense categories. Table 3 provides a consolidated reference used throughout the following sections.

Table 3: GL account codes used by the pipeline

| GL Account | Description |
|---|---|
| 400000 | External Sales - revenue from sales to external customers |
| 400028 | Intercompany Sales - internal transfers between JBS business units |
| 400350 | Gross Sales - V&V sell and buy-back (Farming Operations) |
| 450050 | Trade Allowance Expense (3.Meat Sales and 6.Laverton Processing) |
| 500031 | External Meat Purchase Cost of Sales (3.Meat Sales) |
| 500192 | Freight Out / Domestic Freight Expense |
| 500609 | Freight Ocean / Export Freight Expense |
| 510157 | Cost of Sales - V&V sell and buy-back (Farming Operations) |
| 510167 | Intercompany pig purchases / meat trade cost of sales |
| 650301 | Packaging Expense |
| 650303 | Pallet Expense |

| | |
|---|---|
| 650311 | Packaging and Margin Expense (special contracts: Woolworths, DRJ, Meat Sales) |
| 730010 | Storage / Frozen Stock Holding Expense |

## Revenue Account Logic

### Account Skeleton

For each Business Unit, the first step builds a complete set of account rows from Master Data. This skeleton defines the shape of the output - every possible GL + Cost Center + Customer combination - before any volume is joined. Using `crossing(Period = 1:12)` ensures one row per month for each account, so months with zero volume still appear in the output rather than being silently absent.

```
account_rows <- customer_data %>%
  filter(BU == "1.Farming Operations") %>%
  left_join(gl_cc_data, by = "BU") %>%
  crossing(Period = 1:12)   # ensures all 12 months are present
```

### Volume Join

Volume is joined onto the skeleton using a composite key. For Farm tab rows, the join key is SOP Unicode + Abattoir + Month + Tab:

```
account_rows %>%
  left_join(
    volume_farm_raw,
    by = c("SOP Unicode" = "Unicode", "Abattoir", "Month", "Tab")
  )
```

### Value Calculation

The formula applied to each row depends on the floor type. Table 4 shows the complete set of floors, their value formulas, and the price source used by the pipeline.

Table 4: Value calculation formula and price source by revenue floor

| Floor | Value Formula | Price Source |
|---|---|---|
| Farm | Total Weight × Carcass Price | Carcass Price s |

| | | |
|---|---|---|
| Kill | Units × Kill Fee | Kill Fee sheet - |
| Bone | Units × Bone Fee | Bone Fee sheet |
| Boxed Meat | Total Weight × Item Price | BM PRice shee |
| Offal (Edible / Inedible) | Total Weight × Item Price | BM PRice shee |
| Commodity | Total Weight × Item Price | BM PRice shee |
| Meat Sales (VA) | Total Weight × Price (embedded in SOP Volumes) | Price column i |
| Meat Trade | Total Weight × Item Price (remapped from Laverton bone data) | BM PRice shee |

A key distinction is that Meat Sales (VA) prices come from a `Price` column embedded directly in SOP Volumes rather than from `Prices.xlsb`. This means a missing or zero price in the SOP file silently produces a zero revenue row - one of the failure modes captured by the Gap Check (Section 4). For Farm, Kill, and Bone floors the formula is:

```
mutate(Value = case_when(
  Tab == "FARM" ~ Total_Weight * Carcass_Price,
  Tab == "KILL" ~ Units        * Kill_Fee,
  Tab == "BONE" ~ Units        * Bone_Fee
))
```

The table below illustrates the expected output for one representative Business Unit after all three steps:

Table 5: Illustrative revenue calculation output for one Business Unit

| Tab | Units | Weight (kg) | Rate | Value ($) |
|---|---|---|---|---|
| FARM | 120 | 18500 | $3.25/kg | 60125 |
| KILL | 120 | 18500 | $45.50/hd | 5460 |
| BONE | 85 | 12000 | $28.00/hd | 2380 |

### Revenue Account Coverage

The pipeline covers two distinct GL accounts for most Business Units:

- **GL 400000** - sales to external customers
- **GL 400028** - intercompany sales between JBS sites

Table 6: Revenue account coverage by site and Business Unit

| Site | Business Unit |
|---|---|
| Corowa | 1. Farming Operations |
| Corowa | 2. Meat Processing - Kill Floor |
| Corowa | 2. Meat Processing - Boning Room |
| Corowa | 2. Meat Processing - Boxed Meat |
| Corowa | Offal (Edible & Inedible) |
| Corowa | Commodity Bone |
| Corowa | Meat Trade (receives Laverton volume under Corowa account) |
| Laverton | 6. Laverton Processing - Kill Floor |
| Laverton | 6. Laverton Processing - Boning Room |
| Laverton | 6. Laverton Processing - Boxed Meat |
| Laverton | Offal (Edible shipped to Corowa via Lineage cold store) |
| Laverton | Commodity Chilled & Frozen |
| Laverton | Sow Bone |
| Cross-site | 3. Meat Sales (VA / Value-Add) |

### Special Revenue Conditions

Several revenue accounts require adjustments or overrides beyond the standard weight-times-price formula.

**Laverton Lineage Discount ($0.51/kg)**

Laverton ships three product types - Edible Offal, 6Way, and Commodity Frozen - to the Lineage cold store, which charges a combined storage and handling fee of $0.51/kg. This amount is subtracted from the calculated revenue value for each of these three lines after the standard formula is applied. The discount is the primary reason Corowa and Laverton offal and commodity accounts are maintained as separate calculation paths rather than a shared block.

**Laverton Edible Offal - Intercompany Classification (GL 400028)**

Laverton consolidates its edible offal with Corowa and sells it under Corowa's name. Because the transfer is internal rather than an arm's-length sale, the revenue is recorded under GL 400028 (Intercompany Sales) rather than GL 400000 (External Sales). This distinction is applied during the skeleton build by assigning Laverton Edible Offal rows a different GL account from all other Laverton product lines.

**Corowa Commodity - 400000 vs 400028 Volume Split**

Corowa's Commodity Bone revenue is distributed across two GL accounts. The GL 400028 (Intercompany) volume is sourced from Procurement rows in the Meat Sales tab of SOP Volumes - these represent internal purchases made by the VA business unit. The GL 400000

(External) volume is the remainder: total Commodity Bone weight minus the 400028 internal portion. This split ensures that commodity sold internally is not double-counted as external revenue.

**Laverton Kill - Temporary Customer Exclusion**

A temporary filter removes customers whose names contain "6WAY" or "COMMODITY" from the Laverton Kill tab. These rows are excluded while the correct account mapping for those customer groups is under review. Retaining them would produce duplicated volume across Kill and Bone account categories.

## Expense Account Logic

Expense accounts follow the same skeleton-join-calculate pattern as revenue. The key difference is the rate source: instead of a price per unit, rates are joined from external files (Prices.xlsb packaging sheet, Trade Allowance, and Rates & Allocation). The value formula is consistently:

$$\text{Value} = \text{Total Weight (kg)} \times \text{Rate (\$/kg)}$$

Table 7: Expense categories, GL accounts, and rate sources

| Category | GL Account | Rate Source |
|---|---|---|
| Packaging & Pallet | 650301 / 650303 / 650311 | Prices.xlsb - Packaging VA sheet |
| Trade Allowance | 450050 | Trade Allowance.xlsx |
| Storage | 730010 | Rates & Allocation.xlsx (Sheet2) |
| Freight | 500192 / 500609 | Rates & Allocation.xlsx (Sheet2) |

Table 8: Illustrative expense calculation output for one Business Unit

| Category | Weight (kg) | Rate ($/kg) | Value ($) |
|---|---|---|---|
| Packaging | 18500 | 0.12 | 2220 |
| Trade Allowance | 18500 | 0.05 | 925 |
| Storage | 18500 | 0.08 | 1480 |
| Freight | 18500 | 0.15 | 2775 |

### Packaging Variants

Packaging rates in `Prices.xlsb` are structured around BOM (Bill of Materials) codes and variant numbers. The correct variant to use differs by product type, as shown in Table 9.

Table 9: Packaging VARIANT selection by product type

| Product Type | VARIANT Used |
|---|---|
| Edible Offal (Corowa and Laverton) | VARIANT 3 - with fallback to VARIANT 4 if price is missi |
| 6 Way and Commodity (Corowa and Laverton) | VARIANT 4 directly (no fallback) |
| VA / 3.Meat Sales | VARIANT 2 |

For Edible Offal, prices are first looked up from VARIANT 3. If the result is `NA` or zero-weight, the pipeline falls back to VARIANT 4. This fallback is implemented by taking the grouped maximum across both variants:

```
price_lookup <- packaging_price_data %>%
  filter(VARIANT %in% c(3, 4)) %>%
  mutate(BOM = as.character(BOM)) %>%
  group_by(BOM) %>%
  summarise(PACKAGING = suppressWarnings(max(PACKAGING, na.rm = TRUE)),
            .groups = "drop") %>%
  mutate(PACKAGING = if_else(is.infinite(PACKAGING), NA_real_, PACKAGING))
```

### Laverton Sow Packaging - Temporary Hardcoded Rate

Sow packaging prices are currently absent from the `Packaging VA` sheet (VARIANT 4) of `Prices.xlsb`. As a temporary fix, the pipeline applies a hardcoded rate of $0.095/kg to all Laverton Sow packaging rows unconditionally until the source data is updated.

### Freight Cost Allocation (GL 500192 and GL 500609)

Freight is split between Domestic (GL 500192) and Ocean/Export (GL 500609) for each product group. The pipeline uses a many-to-many join to assign the correct rate to each specific export product type automatically - replacing the prior manual approach of averaging a single rate across all export types, which was acknowledged to be imprecise.

Allocation rates come from `Rates & Allocation.xlsx`, with different sheets used depending on the product category:

- **3.Meat Sales Freight Out (500192-10001814)**: Uses Sheet1 (VA allocation) - a $/kg rate and percentage weight split by Brand and State (Domestic vs Export). Volume source is the VA sales revenue data.
- **Corowa Edible Offal**: Domestic (500192-10001816) and Ocean (500609-10001816) - uses Sheet2 Offal allocation, with a Proportion and $/kg rate split by freight type.
- **Corowa 6 Way**: Domestic (500192-10501816) and Ocean (500609-10501816) - uses Sheet2 6 Way allocation.
- **Corowa Commodity**: Domestic (500192-10501816) and Export (500609-10501816) - uses Sheet2 Commodity allocation.
- **Corowa Meat Trade**: Domestic (500192-10501816) and Ocean (500609-10501816) - uses Sheet2 allocation mapped by Service field (Edible Offal, Commodity, 6 Way).

All freight values are negated (expense).

### Export Carcass Packaging (GL 650301-10201854)

Laverton charges a fixed fee of $12.30 per head for export carcass packaging destined for Singapore and Malaysia. This applies to customers where `Floor == "Export Carcass"` and `Customer Group == "Export"`. The volume source is Kill service rows from `volume_farm_raw`, grouped by SOP Unicode and Month. The result is negated (expense).

### Freight In (GL 500195) - Not Yet Implemented

Freight In accounts exist in the Master Data for all three sites (500195-10001812 for Farming Operations, 500195-10001816 for 2.Meat Processing, 500195-10501854 for 6.Laverton Processing). Freight In customers in `customer_data` are included in the farm revenue path, but these accounts are **not currently present in the final pipeline output** - they have not yet been broken out as separate 500195 rows. This is a known gap for a future development cycle.

### Special Customer Contracts

Several customers deviate from the standard pipeline logic due to contractual or structural requirements.

### Woolworths and DRJ (Margin Contracts)

Woolworths and DRJ operate under fixed-fee packaging contracts that also carry a margin component, requiring distinct GL treatment. Both customers are sourced exclusively from the Boning Room floor (filtered on Tab == “Bone” and Floor == “Bone”), unlike standard Boxed

Meat lines. Their expense values are computed on bone-out weight, applying two sequential yield conversions: a hot-to-cold factor of 90.9% followed by a cold-to-meat factor of 88%.

The contract parameters differ by customer:

| Customer | Packaging Fee | Margin Rate |
|---|---|---|
| Woolworths | \$0.137 / kg | 45.26% |
| DRJ | \$0.15 / kg | 10.00% |

Both customers produce two GL rows per period under cost Center 11901816. Packaging cost is posted to GL 650301 as a positive expense; the margin is posted to GL 650311 and negated. This contrasts with standard boning room lines, which record pallet costs to GL 650303 rather than carrying a margin account at all.

A further pricing rule applies to Woolworths specifically: the `FCOROWAWoolworths` customer record inherits its carcass price from `FEXTERNALBig River Pork - Coles` during the farm price join. This inheritance is applied as a targeted override before the standard price lookup, ensuring the correct benchmark price flows through to the contract margin calculation.

### VA Brand Allocation

Value-Add products sold through 3.Meat Sales are distributed across brands using percentage weights stored in Master Data. These proportions are normalised on load to ensure they always sum to 100% within each brand group, guarding against rounding drift in the source file:

```
VA_allocation_data <- VA_allocation_data %>%
  filter(!is.na(Brand)) %>%
  mutate(`%` = `%` / sum(`%`), .by = Brand)
```

### Cost of Sales Accounts

In addition to the packaging and freight expenses above, the pipeline generates several cost of sales rows that offset revenue accounts. These cover four distinct cost categories.

#### V&V Sell and Buy-Back (GL 400350 and GL 510157 - Farming Operations)

The V&V (Vertically integrated Ventures) arrangement involves Farming Operations selling live pigs to an external party and buying them back as processed product. The pipeline filters the `FLAVERTONNew Team` SOP Unicode from customer data, then calculates value from external VA procurement rows (External vendor, Procurement service) joined to the carcass price ("Other Transactions" row). Two paired rows are produced per period, both under cost Center 10001812:

- **Gross Sales** (GL 400350): the sell-side value, negated as a revenue offset
- **Cost of Sales** (GL 510157): the buy-back cost

**Intercompany Pig Purchase from Farm (GL 510167 - Corowa and Laverton)**

This account captures the cost that Meat Processing pays to Farming Operations for pigs transferred internally. The pipeline filters non-External farm volume rows for Corowa and Laverton unicodes respectively. For Corowa, three expense row types are produced per customer per period:

- **Buy**: Total Weight × Carcass Price (Farm tab)
- **Kill**: Units × Kill Fee (Kill tab, averaged across COROWA6WAY and COROWACOMMODITY unicodes)
- **Bone**: Units × Bone Fee (Bone tab, averaged across COROWA6WAY and COROWA-COMMODITY unicodes)

For Laverton, a single row type is produced: Total Weight × Carcass Price (Farm tab only). Output rows use GL 510167 under cost Center 10501816 (Corowa) and 10501854 (Laverton) respectively.

> **Note:** A known mismatch between customer data and GL & CC accounts exists in this section and is still under investigation.

**Meat Trade Cost of Sales (GL 510167-10501816 - 2.Meat Processing)**

Corowa records an intercompany cost of sales for Laverton products that are transferred to Corowa as Meat Trade. The pipeline filters `Customer Group == "Meat Trade"` from 2.Meat Processing customer data and creates a corresponding cost row under GL 510167, cost Center 10501816.

**Meat Sales Cost of Sales - Internal and External (3.Meat Sales)**

The Meat Sales business unit records two cost of sales rows per period, both filtered on `Customer Group == "Meat Purchase"` from the 3.Meat Sales customer data:

- **Internal purchases** (GL 510167-10001814): cost of meat sourced from within JBS
- **External purchases** (GL 500031-10001814): cost of meat sourced from external suppliers

### Trade Allowance (GL 450050)

Trade allowances are negotiated rebates paid to customers, recorded as a separate expense line rather than as a reduction in the sale price.

**3.Meat Sales (GL 450050-10001814)**: The pipeline joins VA revenue data with `Trade Allowance.xlsx` on Brand and Period. The allowance for each customer is calculated as:

$$\text{Allowance} = (\text{Trade Spend \%} \times \text{Revenue Value}) + \text{Trade Spend Value}$$

Results are summed by customer and negated before posting (expense).

**6.Laverton Processing (GL 450050-10201854)**: A fixed rebate of $5.80 per head is applied to `BE Campbells Bacon` and `B.E. Campbell` customers. Volume is sourced from Kill service rows in `volume_farm_raw`. The result is negated (expense).

### Storage Expense (GL 730010)

Storage costs are calculated for frozen stock held at Lineage and other cold stores. Each account produces two cost rows per period:

- **Cost In**: Total Weight × $0.14/kg
- **Cost Out**: Total Weight × $0.08/kg (Total Weight is recorded as 0 in the output)

The accounts covered are:

Table 11: Storage expense accounts and notes

| Account | GL-CC Key | Note |
|---|---|---|
| Corowa Edible Offal FRZ | 730010-10001816 | Frozen offal only (filtered by Storage == 'FROZEN') |
| Corowa Commodity FRZ | 730010-10501816 | |
| Corowa 6 Way FRZ | 730010-10501816 | |
| Corowa Meat Trade | 730010-10501816 | Uses combined Laverton volume from the Meat Trade revenue |
| Laverton Sow FRZ | 730010-10501854 | |
| Value Add / 3.Meat Sales | 730010-10001814 | Currently set to \$0 as a temporary adjustment |

In addition, three **Stock On Hand** rows apply a fixed monthly hold volume multiplied by a hold rate of $0.01/kg:

- **Corowa Stock On Hand** (730010-10501816): 1,100,000 kg/month
- **Laverton Stock On Hand** (730010-10501854): 200,000 kg/month
- **Value Add Stock On Hand** (730010-10001814): 60,000 kg/month

All storage and stock-on-hand values are negated (expense).

## Cross-Site Logic

Several accounts involve volume flowing between sites.

### Meat Trade (Laverton → Corowa)

Corowa's Boxed Meat section includes a Meat Trade category that consolidates three Laverton product lines sold commercially under Corowa's name. Source rows are drawn from Laverton bone data and remapped so that Corowa is the selling entity (`Abattoir = COROWA`, `GL Account = 400000`, `Cost Center = 10501816`). The three product types and their source filters are:

Table 12: Meat Trade product types and their Laverton source filters

| Meat Trade Type | Laverton Source Filter |
|---|---|
| 6 Way | Run == "6 WAY" |
| Commodity Frozen | Run == "COMMODITY BONE" and Storage == "FROZEN" |
| Edible Offal | Type IN ("OFFALS", "BACON CUTS") and Run == "TOTAL KILL" |

The join to the customer account table uses the `Service` field as the key (values: `6 Way`, `Commodity`, `Edible Offal`). The $0.51/kg Lineage discount is **not** applied to these Meat Trade rows - it is already deducted within the separate Laverton-side Edible Offal, 6Way, and Commodity Frozen account blocks, so applying it again here would result in double-counting.

### Laverton Edible Offal

Edible offal produced at Laverton is shipped to the Lineage cold store and sold under Corowa's name. From a GL perspective this is classified as an intercompany transfer (GL 400028) rather than an external sale (GL 400000), because the transaction is between JBS business units rather than with an outside customer. The $0.51/kg Lineage handling fee is subtracted from the calculated value at the Laverton level. This account is maintained as a separate Laverton calculation path - rather than being absorbed into the Corowa offal block - specifically to isolate the Lineage discount without affecting Corowa's external offal revenue.

---

## Quality Assurance: Gap Check

### Motivation

The Gap Check was built incrementally during pipeline development. Each check in the tool represents a real failure that was discovered during testing - a scenario where the pipeline produced a wrong or missing value without any visible error message. By the end of development,

the Gap Check had captured eight distinct failure modes, covering both revenue and expense accounts.

> Unlike manual Excel review, R pinpoints the exact item, month, and site instantly - a major reduction in the time required to diagnose and fix data issues.

## Output Structure

The Gap Check produces `Gap_Check.xlsx` with two sheets.

**Sheet 1 - SOP vs DB** contains two tables. The first is a weight reconciliation comparing SOP raw weight against the weight that entered the financial database for each of the twelve product categories:

Table 13: Weight reconciliation categories in the Gap Check (Sheet 1)

| Category |
|---|
| Corowa - Offal (Inedible + Edible) |
| Laverton - Offal (Inedible + Edible) |
| Corowa - 6 Way |
| Laverton - 6 Way |
| Laverton - Sow |
| Corowa - Commodity (400000 + 400028) |
| Laverton - Commodity Chilled |
| Laverton - Commodity Frozen |
| Meat Trade - 6 Way (Laverton → Corowa) |
| Meat Trade - Commodity Frozen (Laverton → Corowa) |
| Meat Trade - Edible Offal (Laverton → Corowa) |
| VA / Meat Sales |

A non-zero gap in any category indicates volume that was present in the SOP source but failed to enter the database - typically because of a missing price, an unrecognised identifier, or a classification field left blank. The second table on this sheet is an Issues Summary listing all identified issue types, the number of rows affected, the weight affected in kg, and a suggested fix.

**Sheet 2 - All Issues** combines every failing row across all checks into a single table, with columns for Revenue Related Issue, Expense Related Issue, Abattoir, Tab, Item Code, Item Description, Run, Storage, Type, Group, SOP Unicode, Customer name, Units, and Total Weight. This sheet is designed for direct hand-off to the data owner responsible for fixing the source file.

## Checks Implemented

Table 14: Gap Check: all eight revenue checks with causes and fixes

| # | Check | What It Catches | Recommended Fix |
|---|---|---|---|
| 1 | Item Code not found in price reference (BM PRice sheet) | New product not yet added to the price reference file | Add Item Code to BM PRice sheet in Prices.xlsb |
| 2 | Commodity Bone: missing or invalid Storage classification | Missing Chilled / Frozen label in SOP source data | Fill Storage column (CHILLED / FROZEN) in SOP Volumes |
| 3 | Farm FARM tab: no Carcass Price match | Customer volume with no matching Carcass Price for that period | Add Unicode + Abattoir + Period to Carcass Price sheet |
| 4 | Farm BONE tab: no Bone Processing Fee match | Customer volume with no matching Bone Fee | Add Unicode + Abattoir + Period to Bone Fee sheet |
| 5 | Farm KILL tab: no Kill Fee match | Customer volume with no matching Kill Fee | Add Unicode + Abattoir + Period to Kill Fee sheet |
| 6 | VA Sales: missing or zero price in SOP data | Sale record with blank or zero price in the SOP file | Fill Price column in SOP Volumes (VA tab) |
| 7 | VA Sales: Item Code not in Brand Mapping | Item sold under VA Sales but not yet added to VA Brand mapping in Master Data | Add Item Code to VA sheet in Master Data |
| 8 | Farm: same SOP Unicode mapped to multiple Item Codes | Data entry error in SOP: one Unicode row associated with multiple Item Codes | Correct Unicode or Item Code mapping in SOP Volumes |

In addition to the eight revenue checks, the Gap Check covers expense-side packaging and pallet price gaps for five product categories. These are flagged separately in the Issues Summary with their own Expense Related Issue labels:

Table 15: Gap Check: expense-side packaging and pallet price checks

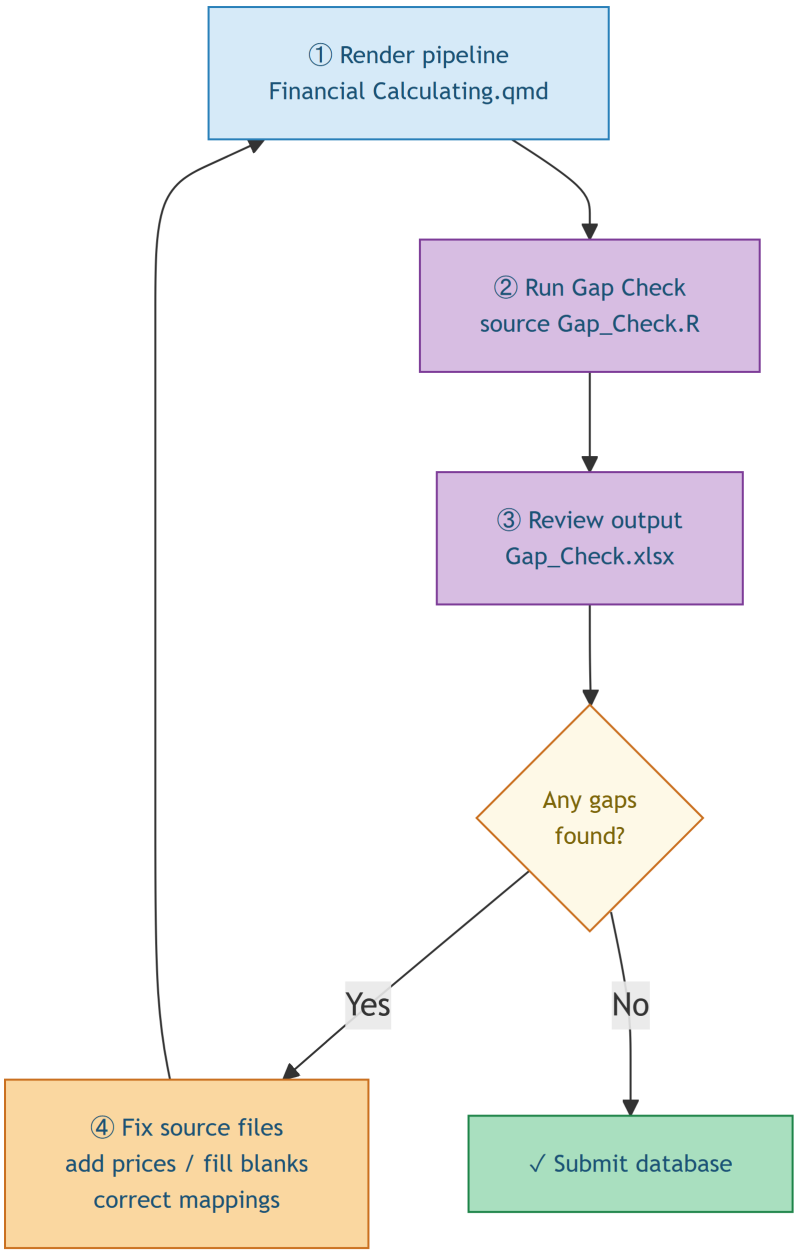
| Expense Check | What It Catches | Recommended Fix |
|---|---|---|
| VA Sales: Missing Packaging Price | VA item whose BOM code has no entry in the Packaging VA sheet | Add Item Code to packaging price data |

| | | |
|---|---|---|
| Laverton Edible Offal: Missing Packaging Price | Edible offal item with no packaging price in VARIANT 3 or 4 | Add Item Code to packaging price data |
| Laverton Edible Offal: Missing Pallet Price | Edible offal item with no pallet price in VARIANT 3 or 4 | Add Item Code to pallet price data |
| Laverton 6 Way: Missing Packaging Price | 6 Way item with no packaging price in VARIANT 4 | Add Item Code to packaging price data |
| Laverton Sow: Missing Packaging Price | Sow item with no packaging price (triggers the hardcoded \$0.095/kg fallback) | Add Item Code to Packaging VA sheet (VARIANT 4) |
| Laverton Commodity: Missing Packaging Price | Commodity item with no packaging price in VARIANT 4 | Add Item Code to packaging price data |

## Usage Workflow

The Gap Check is designed to be run immediately after every pipeline render. The standard workflow is:

1. Render `Financial Calculating.qmd` - this produces the financial database output and leaves all pipeline data frames in the R environment.
2. Source `Gap_Check.R` - this reads those data frames and writes `Gap_Check.xlsx`.
3. Open `Gap_Check.xlsx` and review: any non-zero `Rows Affected` value in the Issues Summary table identifies data that needs to be fixed in the source files before submission.
4. Fix the source file issues, re-render the pipeline, and re-run the Gap Check.




This cycle ensures the submitted database is as complete as possible, with any remaining gaps explicitly documented rather than silently absent.

---

# Results and Evaluation

## Before and After Comparison

Table 16: Excel manual process vs R pipeline comparison

| Dimension | Manual Excel (Before) | R Pipeline (Now) |
|---|---|---|
| Time to build each period | Several hours of manual work per period | One-click render, under 10 minutes |
| Adding a new Business Unit | Copy, paste and reformat an entire sheet manually | Add a filter condition and a join - minutes of code change |
| Handling a rule change or new contract | Manually update every affected cell across all months | Edit one conditional or rate value, re-run |
| Data error detection | Manual row-by-row scanning - easy to miss | Gap Check pinpoints exact item, month, and site instantly |
| Risk of human error | High - copy-paste mistakes and missed rows go unnoticed | Low - identical logic applied on every run |
| Freight cost allocation | Cost averaged across all export types - imprecise and acknowledged to be incorrect | Many-to-many join assigns the exact rate to each specific export type automatically |
| Quality assurance | Manual spot-check only | Automated Gap Check runs after every build |

## Account Coverage Achieved

The pipeline covers the full set of accounts required across both sites:

- **Revenue**: 14 distinct Business Unit / site combinations, each with GL 400000 and/or GL 400028 entries across up to 12 months
- **Expense**: four expense categories (Packaging, Trade Allowance, Storage, Freight) applied across all applicable BUs, with several GL variants per category

The Gap Check weight reconciliation table provides a quantitative measure of pipeline completeness: the closer the DB Weight column is to the SOP Raw Weight for each category, the more complete the output. Any remaining gap is documented with an identified cause.

### Data Quality Improvement

Beyond the core pipeline output, the development process itself improved data quality in the source files. Each Gap Check failure during testing triggered a fix - either in the pipeline logic or, more often, in the source file (adding a missing price, correcting a spelling, filling a blank storage field). The iterative nature of development meant these fixes accumulated over multiple runs, resulting in cleaner source data going forward.

---

## Discussion and Limitations

### Current Limitations

There are quite a lot of limitations to the project due to the limitation of time and the limited skill during the development of this project:

1. **Hardcoded file paths.** The pipeline reads files from absolute OneDrive paths specific to the development machine (`C:/Users/ATran2/OneDrive - JBS/...`). Running the pipeline on any other machine requires updating these paths. A configuration file or relative path approach would make the pipeline portable across the team.
2. **Sensitivity to source file structure.** The pipeline depends on specific sheet names, column positions, and value formats in each source file. If a source file's structure changes - for example, if a sheet is renamed, a column is added, or the SOP identifier values change - the pipeline will fail silently or produce incorrect output. Adding schema validation checks at the top of the pipeline would catch these changes before they propagate.
3. **Growing conditional complexity.** Each new customer contract with special pricing or GL rules requires additional conditional logic in the pipeline. Over time, this makes the codebase harder to maintain and reason about. A more scalable approach would be to encode all special cases in the Master Data file itself (as a rules table), rather than in the R code.
4. **No automated scheduling.** The pipeline must be manually triggered each reporting period. While the one-click render is a major improvement over manual Excel work, further automation (e.g., a scheduled RScript run triggered on file update) would reduce dependency on the analyst being available.
5. **Prices.xlsb format.** Binary `.xlsb` files are not natively supported by all R packages. The pipeline uses `openxlsx2::wb_to_df()` which handles this format, but it is slower and occasionally less robust than reading `.xlsx` files. If the Sales Team can export Prices as `.xlsx`, this dependency would be removed.

6. **Limited R adoption within Rivalea.** R is not widely used or supported at JBS Pork Australia at the time of writing. This means the pipeline currently depends on the owner of the project being available to run and maintain it. A colleague unfamiliar with R would face a steep setup cost - installing R, RStudio, and the required packages - before being able to execute the pipeline at all. This limits the pipeline's resilience as a business tool if the owner of the project or someone with R skills is unavailable.
7. **Single-file codebase.** The entire pipeline is contained in one `.qmd` file exceeding 5,000 lines of code. While this was practical during development, it is not a maintainable structure for the long term. Navigating, debugging, or extending any one Business Unit requires scrolling through a large undivided file, and a mistake in one section can unintentionally affect another.

## Future Improvements

Several extensions would increase the utility and robustness of the pipeline:

1. **Parameterised paths**: read file locations from a single config file, allowing the pipeline to be run on any machine without code changes.
2. **Schema validation on load**: check that expected columns and sheets exist before the pipeline proceeds, and produce a clear error message if not.
3. **Rules-driven special cases**: move contract-specific GL rules from R code into Master Data, so the finance team can update them without touching the pipeline.
4. **Shiny front-end**: a simple Shiny app wrapping the pipeline would allow non-R users to trigger a run, review the Gap Check, and download the output without opening RStudio.
5. **Unit tests**: `testthat` tests on key calculation functions (value = weight × rate, skeleton join completeness) would provide confidence that pipeline logic changes have not broken existing accounts.
6. **Modular code restructure**: the current single-file structure should be split into separate modules, one per Business Unit or logical group. The proposed naming convention - `<bu>_<gl_account>_<cc_key>.R` (e.g., `offal_glaccount_cckey.R`) - would make each module self-describing: given the file name alone, a reader would immediately know which product, GL account, and Cost Center it covers, along with the calculated values it produces. This would dramatically reduce the cognitive load of maintaining or extending any individual account block, and allow multiple people to work on different modules simultaneously without risk of conflict.

---

## Conclusion

This project successfully delivered an automated R pipeline that replaces the manual, error-prone Excel-based workflow for financial account creation at JBS Pork Australia (Rivalea). The pipeline reads five source files from a shared OneDrive folder, applies the full revenue and expense calculation logic across all Business Units at both the Corowa and Laverton sites, and produces a structured financial database ready for import.

The companion Gap Check tool was an equally important contribution: by making data integrity issues visible and actionable immediately after each pipeline run, it addresses the core risk of the original workflow - that errors would propagate silently into the submitted output.

The most significant practical limitations are the limited adoption of R within Rivalea, which means the pipeline currently depends on the owner of the project to run and maintain it, and the single-file codebase structure, which at over 5,000 lines is difficult to navigate and extend. Hardcoded file paths and growing conditional complexity from special contract logic are further constraints. The most important future step is a modular restructure - splitting the pipeline into per-account modules named by GL account and Cost Center key - which would make the codebase maintainable by others and easier to extend as new Business Units or contracts are added. The foundation built here - a reproducible, version-controlled, auditable pipeline - provides a substantially better basis for financial reporting than the manual process it replaces.

---

## References

The following R packages were used in this project:


- Wickham, H. et al. (2023). *dplyr: A Grammar of Data Manipulation*. R package version 1.1.4. https://dplyr.tidyverse.org
- Wickham, H. and Bryan, J. (2023). *readxl: Read Excel Files*. R package version 1.4.3. https://readxl.tidyverse.org
- Schauberger, P. and Walker, A. (2023). *openxlsx: Read, Write and Edit xlsx Files*. R package version 4.2.5. https://CRAN.R-project.org/package=openxlsx
- Hester, J. et al. (2024). *openxlsx2: Read, Write and Edit xlsx Files*. R package version 1.1. https://janmarvin.github.io/openxlsx2/
- Wickham, H. (2023). *stringr: Simple, Consistent Wrappers for Common String Operations*. R package version 1.5.1. https://stringr.tidyverse.org
- Grolemund, G. and Wickham, H. (2011). *lubridate: Make Dealing with Dates a Little Easier*. R package version 1.9.3. https://lubridate.tidyverse.org

- Wickham, H. and Girlich, M. (2022). *tidyr: Tidy Messy Data*. R package version 1.3.0. https://tidyr.tidyverse.org
- Xie, Y. (2024). *knitr: A General-Purpose Package for Dynamic Report Generation in R*. R package version 1.45. https://yihui.org/knitr/
- Zhu, H. (2024). *kableExtra: Construct Complex Table with 'kable' and Pipe Syntax*. R package version 1.3.4. https://haozhu233.github.io/kableExtra/